



ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ
ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

EEM-272
MAKİNE ÖĞRENMESİNE GİRİŞ DERSİ
SUNUM RAPORU

DERS SORUMLUSU
Dr. Öğr. Üyesi Ali ŞENTÜRK

“Yüz Tanıma ve Metrik Öğrenme: One shot learning, Siamese Networks, Triplet Loss ve Embedding Mantığı Üzerine Bir İnceleme”

HAZIRLAYAN

Defne Benliay
2412705020

1. GİRİŞ

Yüz tanıma (Face Recognition), bir dijital imgeden biyometrik özneliklerin çıkarılarak, bu verilerin önceden tanımlanmış bir kimlik havuzuyla eşleştirilmesi sürecidir [1- 7]. Bu biyometrik yöntemler arasında "Yüz Tanıma" (Face Recognition), kullanıcıdan aktif bir temas gerektirmemesi ve mevcut kamera altyapılarıyla entegre çalışabilmesi nedeniyle en çok tercih edilen alanlardan biri haline gelmiştir. Geleneksel yöntemler görüntüyü pikseller bazında eşleştirmeye çalışırken, modern derin öğrenme yaklaşımları yüzü anlamsal bir düzlemde analiz eder. Sunumda ve literatürde belirtildiği üzere, bu süreç iki temel aşamadan oluşur: İlk olarak görüntüdeki yüzün konumunun saptandığı "Yüz Tespiti" (Face Detection) ve ardından bu yüzün kime ait olduğunun belirlendiği "Yüz Tanıma" (Face Recognition). Yüz Tespiti (Detection), "Görüntüde bir insan yüzü var mı?" sorusuna yanıt ararken ve koordinat belirlerken; Yüz Tanıma (Recognition), tespit edilen bu bölgenin kime ait olduğunu belirleme işleviyle ilgilenir. Bu süreçte pikseller, "Embedding" adı verilen ve genellikle 128 boyuttan oluşan matematiksel özetlere (dijital parmak izlerine) dönüştürülür [2 - 6].

Geleneksel derin öğrenme modelleri, nesneleri kategorize etmek için son katmanda "Softmax" fonksiyonu kullanır, fakat bu yöntem yüz tanımada başarısızdır. Bir bankanın veya üniversitenin sistemine her yeni kişi eklendiğinde modelin milyonlarca parametresini yeniden eğitmek (re-training) hesaplama açısından imkansızdır. Bu kısıtlamalar, sistemleri "Açık Küme" (Open-set) problemlerini çözebilen, yani daha önce görmediği bir yüzü bile mesafe üzerinden ayırt edebilen Metrik Öğrenme (Metric Learning) ve Siyam Ağları mimarilerine yöneltmiştir [3].

Çalışma için iki adet proje gerçekleştirilmiş, bunlara ait bilgiler Ek-1 ve Ek-2 de verilmiştir. Aynı şekilde sayısal olarak kod dizimine [ddbenliay-lang](https://github.com/ddbenliay-lang) github profilinden ve aşağıda verilmiş olan Google Colab linkinden ulaşılabilir.

CIDAR10 Metrik Öğrenme projesi için:

<https://colab.research.google.com/drive/14gYdWFAIKKx5hGBiNFXL3bfNMmC0Ux3U?usp=sharing>

Triplet Loss Projesi için:

https://colab.research.google.com/drive/1rvNqiYHYnFzI86ifLn70FNVqr7G_wQU-?usp=sharing

2. TEMEL KAVRAMLAR

2.1 Siamese Networks

Siyam ağıları, birbirinin tıpatıp aynısı olan iki (veya daha fazla) ayrı sinir ağı kanalından oluşur. Buradaki kritik nokta, bu kanalların sadece mimarilerinin aynı olması değil, aynı zamanda ağırlıklarının (weights) da birbiriyle paylaşıyor olmasıdır. Yani, bir kanalda gerçekleşen öğrenme süreci aslında diğer kanalın da öğrenmesini sağlar; bu da ağın her iki girdiyi de aynı "bakış açısıyla" değerlendirmesine olanak tanır.

Temel görev ve işleyişi, iki farklı girdi görüntüsünü aynı anda işleyerek bu girdileri yüksek boyutlu bir vektör uzayına, yani öznitelik (embedding) seviyesine taşımaktır. Sürecin sonunda ağ, şu sorulara yanıt arar:

- Bu iki görüntü birbirine ne kadar benziyor?
- Aralarındaki anlamsal mesafe nedir?

2.2. Embedding

Yüz tanıma sistemlerinde Embedding (Gömme), yüksek boyutlu ham görüntü verisinin (piksellerin), derin öğrenme modelleri aracılığıyla düşük boyutlu, yoğun ve anlamlı bir sayısal vektöre dönüştürülme sürecidir. Bu süreç, karmaşık görsel bilgiyi bilgisayarın hızla işleyebileceği ve kıyaslayabileceği bir "matematiksel parmak izine" indirger. Embedding sürecinin temel bileşenleri ve işleyişi

- **Öznitelik Uzayına Geçiş:** Bir yüz görüntüsü, evrişimli sinir ağıları (CNN) katmanlarından geçirilerek analiz edilir. Sistemin çıktısı olan embedding vektörü, genellikle 128 veya 512 boyutlu bir öznitelik uzayında bir nokta (koordinat) belirtir. Bu dönüşüm sayesinde, milyonlarca pikselden oluşan veri yükü, sadece en ayırt edici bilgileri taşıyan kompakt bir sayı dizisine dönüştürülür.
- **Anlamsal Kodlama:** Vektörü oluşturan her bir sayısal değer, yüzün biyometrik karakteristiklerini (göz yapısı, burun kemiği yüksekliği, elmacık kemiği yerleşimi, çene hattı vb.) soyut birer parametre olarak temsil eder. Model, eğitim aşamasında bu özellikleri birbirinden ayırmayı öğrenir; böylece embedding değerleri rastgele sayılar değil, yüzün geometrik yapısının matematiksel bir özeti haline gelir.
- **Vektörel Uzaklık ve Karşılaştırma:** Yüz doğrulama işlemi, iki farklı görüntünün embedding vektörleri arasındaki geometrik uzaklığın hesaplanması esasına dayanır. Öklid Mesafesi veya Kosinüs Benzerliği gibi metrikler kullanılarak yapılan bu

hesaplama; mesafe ne kadar küçükse, iki görüntünün aynı kişiye ait olma olasılığı o kadar yüksektir.

Öklid Mesafesi (Euclidean Distance): İki nokta arasındaki doğrusal mesafe formülü aşağıda verilmiştir.

$$D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Öklid mesafesi

Formül 1. Öklid Mesafesi Formülü

Kosinüs Benzerliği (Cosine Similarity): Orijinden çıkan iki vektör arasındaki açının kosinüsüdür.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

Formül 2. Kosinüs Benzerliği Formülü

L2 normalizasyonu yapılmış vektörlerde bu iki metrik birbirine doğrudan orantılıdır [4].

$$d^2 = 2 - 2(\cos \theta)$$

Formül 3. Kosinüs Teoremi Formülü

Gelişmiş Embedding modelleri, kişinin farklı ışık koşulları, farklı bakış açıları veya aksesuarlar (gözlük, sakal vb.) altındaki görüntülerinden birbirine çok yakın vektörler üretmeyi hedefler. Bu "ayırt edici güç", sistemin gerçek zamanlı ortamlarda yüksek doğrulukla çalışmasını sağlayan en kritik unsurdur.

2.3 Metrik Öğrenme ve Triplet Loss (Üçlü Kayıp)

Siyam ağlarının eğitiminde kullanılan en efektif mekanizmalardan biri Triplet Loss (Üçlü Kayıp) fonksiyonudur. Bu yöntem, eğitimi sadece "benzer/benzemez" ikilisi üzerinden değil, üçlü bir set üzerinden yürütür:

Anchor (Çapa): Referans alınan kişinin yüzü.

Positive (Pozitif): Aynı kişinin farklı bir fotoğrafı.

Negative (Negatif): Başka bir kişinin fotoğrafı.

Eğitimin amacı, Çapa ile Pozitif arasındaki mesafeyi minimize ederken, Çapa ile Negatif arasındaki mesafeyi maksimize etmektir. Bu, modelin "kişiye özel" bölgeler oluşturmasını sağlar.

2.4 One-Shot Learning (Tek Seferlik Öğrenme)

Geleneksel derin öğrenme modelleri bir sınıfı tanımak için binlerce veriye ihtiyaç duyarken, Siyam ağı One-Shot Learning yeteneği sayesinde bir kişiyi sadece tek bir referans fotoğrafı ile tanıyabilir.

Sistem, yeni bir yüzü "sınıflandırmayı" öğrenmek yerine, o yüzün embedding vektörünü kayıtlı olan "güvenilir" vektörle kıyaslar. Bu, özellikle her gün binlerce yeni kullanıcının eklendiği biyometrik sistemlerde modelin her seferinde yeniden eğitilmesi zorunluluğunu ortadan kaldırır.

3. YÖNTEM / MODEL AÇIKLAMASI

Bu çalışmada ele alınan yüz tanıma sistemi, derin metrik öğrenme (deep metric learning) prensiplerine dayanan ve düşük veri setlerinde yüksek genelleme kapasitesi sunan bir model yapısını temel almaktadır. Sistemin işleyişi; verinin öznitelik uzayına taşınması, ağırlık paylaşımlı ağlar üzerinden karşılaştırılması ve kayıp fonksiyonları ile optimize edilmesi aşamalarından oluşmaktadır.

3.1 Siamese Networks Mimarisi ve Ağırlık Paylaşımı

Önerilen model, iki özdeş evrişimli sinir ağından (CNN) oluşan bir Siyam mimarisi üzerine kurgulanmıştır. Geleneksel sınıflandırma modellerinin aksine bu yapı, girdi çiftlerini bağımsız ancak paralel iki kanal üzerinden işler. Mimarideki en kritik bileşen olan ağırlık paylaşımı, her iki kanalın da aynı öznitelik çıkarım parametrelerini kullanmasını sağlar. Bu durum, girdi çiftleri arasındaki farkın sadece verinin kendisinden kaynaklanmasını garanti ederken, ağırlık öğrenme kapasitesini benzerlik tespiti üzerine yoğunlaştırır.

3.2 Embedding Süreci ve Vektörel Temsil

Modelin temel çıktısı, ham piksel verisini n -boyutlu (genellikle 128 veya 512) bir Embedding (gömme) vektörüne dönüştürmektir. Bu süreçte, görüntüdeki biyometrik özellikler (göz geometrisi, burun yapısı, çene hattı vb.) soyutlanarak matematiksel birer koordinata indirgenir. Oluşturulan bu öznitelik uzayında, aynı kişiye ait görüntüler birbirine yakın noktalara, farklı kişilere ait görüntüler ise uzak noktalara eşlenir.

3.3 Benzerlik Metrikleri ve Karşılaştırma

Elde edilen vektörlerin benzerlik düzeyini saptamak amacıyla Öklid Mesafesi ve Kosinüs Benzerliği metrikleri kullanılmaktadır. L2 normalizasyonu uygulanmış vektörlerde bu iki metrik arasındaki ilişki, trigonometrik bir temele bir formül ile ifade edilir: Bu formül, vektörler arasındaki açı küçüldükçe mesafenin sıfıra yaklaştığını ve sistemin bu iki görüntüyü "aynı kişi" olarak tanımladığını doğrular.

3.4 Triplet Loss ile Model Optimizasyonu

Modelin eğitimi, Triplet Loss (Üçlü Kayıp) fonksiyonu kullanılarak gerçekleştirilir. Bu yöntem, her eğitim adımında bir referans (Anchor), bir aynı sınıftan örnek (Positive) ve bir farklı sınıftan örnek (Negative) kullanarak öznitelik uzayını dinamik olarak düzenler. Kayıp fonksiyonu, pozitif örneği referansa yaklaştırırken negatif örneği güvenli bir marj aralığı kadar uzağa iter. Bu optimizasyon stratejisi, sistemin One-Shot Learning yeteneği kazanmasını sağlayarak, tek bir referans fotoğrafı ile gerçek zamanlı ve yüksek doğruluklu tanıma yapılmasına olanak tanır.

FaceNet mimarisi ve Triplet Loss kullanımını şu adımları izler:

Girdi Seçimi (Batch Selection): Eğitim sırasında "Triplet" adı verilen üçlü gruplar seçilir:

- **Anchor (A):** Rastgele bir kişinin yüzü.
- **Positive (P):** Aynı kişinin farklı bir fotoğrafı.
- **Negative (N):** Tamamen farklı birinin fotoğrafı.

Özellik Çıkarımı (Deep CNN): Üç görüntü de aynı CNN (örneğin Inception veya VGG) yapısından geçirilerek $f(A)$, $f(P)$, $f(N)$ vektörleri elde edilir [3], [4].

Kayıp Fonksiyonu Uygulaması: Model, şu eşitsizliği sağlamaya çalışır:

$$Loss = \sum_{i=1}^N \left[\|f_i^a - f_i^p\|_2^2 - \|f_i^a - f_i^n\|_2^2 + \alpha \right]_+$$

Ağırlık Güncelleme: Eğer negatif örnek, referansa pozitif örnekten daha yakınsa veya aradaki fark α 'dan küçükse hata (loss) yüksek çıkar ve modelin ağırlıkları geriye yayılım (backpropagation) ile güncellenir [5].

4. UYGULAMA ALANLARI

Siyam ağları (Siamese Networks), modern bilgisayarlı görü dünyasında benzerlik ölçümü ve karşılaştırmalı analiz yetenekleriyle çığır açan bir mimari olarak öne çıkmaktadır. Geleneksel derin öğrenme modellerinin aksine, nesneleri doğrudan sınıflandırmak yerine iki farklı girdi arasındaki metriksel benzerliği tespit etmeye odaklanır. Bu benzersiz yapı, sınırlı veriyle yüksek doğrulukta sonuçlar üretmeyi mümkün kılarak "One-Shot Learning" gibi yenilikçi yaklaşımların temelini oluşturur.

Günümüzde bu teknoloji, saniyeler içinde binlerce kareyi analiz edebilen gerçek zamanlı nesne takibi sistemlerinde ve biyometrik güvenlik protokollerinde kritik bir rol oynamaktadır. Özellikle karmaşık veri setlerinde bile ayırt edici öznelikler başarıyla vektörel bir uzaya eşlemesi, adli analizlerden kişi yeniden tanımlama (Re-ID) süreçlerine kadar geniş bir uygulama yelpazesi sunar.

Triplet Loss gibi gelişmiş kayıp fonksiyonlarıyla desteklenen bu ağlar, değişken ışık ve açı koşullarında bile yüksek dayanıklılık sergileyerek dijital güvenliğin standartlarını belirlemektedir. Dolayısıyla Siyam ağları, nesnelerin sadece "ne" olduğunu değil, birbirleriyle "ne kadar benzer" olduğunu anlama kabiliyetiyle yapay zekanın en dinamik dallarından birini temsil eder. Aşağıdaki metin, bu güçlü mimarinin video analizinden adli tıp uygulamalarına kadar uzanan stratejik kullanım alanlarını ve teknik üstünlüklerini detaylandırmaktadır.

Aşağıda bu teknolojilerin kullanım alanları için çeşitli konu başlıkları halinde anlatımlara yer verilmiştir.

Gerçek Zamanlı Nesne Takibi: Siyam ağları, bilgisayarlı görü dünyasında video analizinin temel taşı olan nesne takibinde devrim yaratmıştır. Özellikle SiamFC (Fully-Convolutional Siamese Networks) mimarisi bu alanın öncüsüdür [6 - 1], [7-2]. Çalışma prensibi, bir videonun ilk karesinde kullanıcı tarafından işaretlenen nesne (target template), siyam ağının bir koluna "referans" olarak verilir. Videonun sonraki karelerinde ise geniş bir arama alanı (search region) diğer kola giriş yapılır. Kross-Korelasyon Katmanı olarak iki koldan gelen öznelik haritaları (feature maps) üzerinde çapraz korelasyon işlemi uygulanır. Bu işlem sonucunda bir "skor haritası" (score map) üretilir. Skor haritasındaki en yüksek değer, nesnenin yeni karedeki koordinatını belirler. Geleneksel takipçiler nesneyi her karede yeniden sınıflandırmaya çalışırken, Siyam ağları sadece "benzerlik" arar. Bu sayede modelin tüm ağırlıklarını güncellemeye gerek kalmadan, saniyede 100 kareden (100 FPS) fazla hızla nesne takibi yapılabilir.

Biyometrik Güvenlik: Modern akıllı telefonlardaki yüz tanıma kilitleri (FaceID vb.), Siyam ağlarının One-Shot Learning yeteneğine dayanır. Bu sistemler, kısıtlı donanım kaynaklarında maksimum güvenlik sağlamak üzere tasarlanmıştır [1- 7], [2 - 6]. One-Shot Learning ile klasik modeller bir kişiyi tanımak için binlerce fotoğrafa ihtiyaç duyarken, Siyam ağları sizin kurulum aşamasında verdiğiniz tek bir referans fotoğrafı ile

çalışır. Vektörel uzayda (embedding space) sizin yüzünüzün koordinatı bir kez belirlenir. Dinamik değişimlerde bile iyi sonuçlar verir. İyi eğitilmiş bir Siyam ağı, sakal bırakmanız, gözlük takmanız veya yaşlanmanız gibi durumlarda bile yüzünüzün "anlamsal özünü" (semantic core) korumaya devam eder. Vektörünüz uzayda biraz kaysa bile, hala "sizin bölgenizde" kalır. Liveness Detection (Canlılık Kontrolü) kullanarak güvenlik sistemlerinde Siyam ağları genellikle derinlik haritalarıyla birleştirilir. Amaç, bir fotoğrafla (2D) gerçek bir insan yüzünü (3D) ayırt etmektir. Mesafe metriği burada "gerçek yüz vektörü" ile "sahte yüz vektörü" arasındaki devasa farkı saniyeler içinde saptar.

Adli Tıp ve Kişi Yeniden Tanımlama (Re-ID): Güvenlik kameraları ağında (Multi-camera surveillance) bir şüphelinin takibi, bilgisayarlı görünümün en zorlu problemlerinden biridir. Re-ID sistemleri, Siyam ağlarını ve Triplet Loss mekanizmasını kullanarak bu sorunu çözer [5]. Görüş Açısı ve Işık Değişkenliği kontrolü ile bir kamera kişiyi tepeden, diğeri ise karşıdan görebilir. Siyam ağları, kıyafet dokusu, boy-kilo oranı ve yürüme karakteristiği gibi özellikleri "açıdan bağımsız" (viewpoint-invariant) birer vektöre dönüştürür. Triplet Loss Stratejisi kullanılarak adli analizlerde model; aynı kişiye ait farklı açılardaki görüntüleri (Anchor ve Positive) birbirine yaklaştırırken, ona çok benzeyen başka bir şüpheliyi (Negative) vektör uzayında uzağa iter. Bu sayede "yanlış eşleşme" (false positive) oranı minimize edilir.

Büyük Veri Tabanlarında Arama: Bir suç mahalinden alınan görüntü, şehrin tüm kamera kayıtlarındaki milyarlarca vektörle kıyaslanır. Siyam ağlarının ürettiği 128 boyutlu kompakt vektörler (embeddings) sayesinde, bu devasa arama işlemi milisaniyeler içinde tamamlanır.

5. MİNİ PROJE AÇIKLAMASI

Metrik Öğrenme ve Triplet Loss ile Yüz Tanıma olmak üzere, iki adet mini proje hazırlanmıştır.

5.1.CIFAR-10 ile Metrik Öğrenme

Bu proje kapsamında, derin öğrenme ve metrik öğrenme prensipleri kullanılarak farklı nesne kategorilerini vektör uzayında birbirinden ayıran bir model geliştirilmiştir. Proje, Google Colab ortamında Python dili kullanılarak geliştirilmiştir. Temel kütüphaneler olarak TensorFlow ve Keras mimarisi kullanılmış; veri işleme için NumPy, görselleştirme için ise Matplotlib kütüphanelerinden yararlanılmıştır.

Çalışmada, bilgisayarlı görü literatüründe standart kabul edilen CIFAR-10 veri seti kullanılmıştır. Veri seti; uçak, araba, kuş, kedi, geyik, köpek, kurbağa, at, gemi ve kamyon olmak üzere 10 farklı sınıftan oluşmaktadır. Her bir görüntü 32x32 piksel çözünürlüğünde ve RGB (renkli) formatındadır. Görüntü pikselleri [0, 1] aralığına normalize edilerek modelin daha hızlı yakınsaması sağlanmıştır.

Proje, geleneksel bir sınıflandırma yerine Siamese ağ yapısına dayalı bir benzerlik öğrenme yaklaşımını benimser. Model; üç adet evrişimli (Convolutional) katman, ardından bir Global Average Pooling katmanı ve son olarak bir "embedding" (gömme) uzayı oluşturan yoğun (Dense) katmandan oluşur. Elde edilen vektörler, birim uzunluğa normalize edilerek (Unit Normalization) benzerlik hesaplamaları için hazır hale getirilmiştir.

Eğitim sürecinde her sınıftan rastgele seçilen bir çapa (anchor) ve o sınıfa ait bir pozitif örnek çifti kullanılarak modelin aynı sınıfları birbirine yaklaştırması sağlanmıştır. Eğitimde SparseCategoricalCrossentropy kayıp fonksiyonu ve Adam optimizasyon algoritması tercih edilmiştir. Benzerlik ölçütü olarak ise normalize edilmiş vektörler üzerinde Kosinüs Benzerliği (Cosine Similarity) hesaplanmıştır. Eğitim Performansı: 20 epoch süren eğitim sonucunda, kayıp (loss) değerinin kararlı bir şekilde düştüğü gözlemlenmiştir.

Model, test kümesindeki görüntüler için en yakın komşuları başarıyla tespit edebilmiştir. Örneğin; bir gemi görüntüsü için bulunan en yakın 10 örneğin yine gemi kategorisinden olduğu görselleştirilmiştir. Karışıklık Matrisi (Confusion Matrix) olan performans analizi sonucunda, özellikle "Otomobil" gibi belirgin özelliklere sahip sınıflarda yüksek doğruluk gözlemlenirken; "Kedi" ve "Köpek" gibi birbirine benzeyen hayvan sınıfları arasında yer yer karışıklıklar yaşandığı tespit edilmiştir.

Sisteme yüklenen yeni resimler üzerinde yapılan testlerde, modelin çoğu örnek için başarılı sonuçlar verdiği ancak kendisine tanıtılmamış olan bir görsel için olabilecek en yakın sonuçları verdiği gözlemlenmiştir. Kimi "Kedi" veya "Gemi" görselini doğru kategorize ederken kiminde yanlış cevaplar verebilmektedir.

5.2. Triplet Loss ile Yüz Tanıma Sistemi

Bu proje kapsamında, derin öğrenme yöntemleriyle elde edilen özellik vektörleri kullanılarak, kişilerin kimliklerini tespit edebilen uçtan uca bir yüz tanıma sistemi geliştirilmiştir. Uygulama; Python, TensorFlow/Keras, OpenCV (yüz tespiti için) ve Scikit-Learn (PCA ve k-NN için) kütüphaneleri kullanılarak Google Colab ortamında geliştirilmiştir.

Çalışmada, yüz görüntülerinden ayırt edici "embedding" (gömme) vektörleri çıkaran bir metrik öğrenme modeli kullanılmıştır. Vektör Normalizasyonu olarak modelden çıkan vektörler, birim uzunluğa normalize edilerek (Unit Normalization) benzerlik hesaplamaları için standart hale getirilmiştir. Modelin ürettiği yüksek boyutlu vektörlerin uzaydaki dağılımını analiz etmek için Temel Bileşenler Analizi (PCA) uygulanmıştır. Elde edilen 2 boyutlu grafikte, aynı kişiye ait yüzlerin birbirine yakın kümeler oluşturduğu görülmüştür.

Sistem, sadece benzerlik ölçmekle kalmayıp aynı zamanda bir sınıflandırma katmanı içermektedir: Özellik uzayındaki vektörleri sınıflandırmak için k-En Yakın Komşu (k-Nearest Neighbors) algoritması kullanılmıştır. Model, bilinmeyen bir yüzü, eğitim setindeki en yakın 7 örneğin (n_neighbors=7) çoğunluk sınıfına göre etiketlemektedir. Haar Cascade ile Yüz Tespiti: Kullanıcı tarafından yüklenen fotoğraflar üzerinden yüz bölgesini otomatik olarak ayıklamak için face_cascade.detectMultiScale yöntemi kullanılmıştır.

Tespit edilen yüzler 60x60 veya 32x32 boyutlarına yeniden ölçeklendirilmiş ve normalize edilerek model girişine hazır hale getirilmiştir. Sistem; Elton John, Ben Affleck gibi isimlerin yer aldığı test setinde başarılı sonuçlar vermiştir. Yapılan denemelerde "Predicted name" ile "Actual pred" değerlerinin uyduğu görülmüştür. Doğruluk Oranı: Test kümesi üzerinde yapılan genel değerlendirmede, sistemin yaklaşık %70.8 oranında bir doğruluk (accuracy) ile çalıştığı tespit edilmiştir.

Bazı düşük çözünürlüklü veya aşırı açılı görsellerde tanıma performansının etkilendiği, ancak genel kümeleme başarısının stabil olduğu gözlemlenmiştir.

6. SONUÇ

Bu rapor, yüz tanıma sistemlerinin basit bir "sınıflandırma" probleminden, "çok boyutlu uzayda metrik öğrenme" problemine evrimini tüm matematiksel altyapısıyla ortaya koymuştur.

Geleneksel Softmax yapısının kapalı küme (closed-set) kısıtlamaları, Siyam Ağları ve Triplet Loss ile aşılıarak, modeli yeniden eğitmeye gerek kalmadan yeni kişilerin sisteme eklenebilmesi sağlanmıştır [2 - 6]. Mini projede gerçekleştirilen MTCNN hizalama işleminin ve FaceNet tabanlı vektör çıkarımının, L2 normalizasyonu ve kosinüs benzerliği ile yüksek performanslı bir doğrulama sistemi kurmak için yeterli olduğu görülmüştür.

Endüstrinin geldiği son noktada, Öklid temelli kayıpların yerini küresel açılara odaklanan ArcFace [8] mimarilerinin aldığı tespit edilmiş olup, gelecekteki güvenlik sistemlerinin bu açısal metriklerle tasarlanması gerektiği sonucuna varılmıştır.

7. KAYNAKÇA

- [1] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 1701–1708.
- [2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 815–823.
- [3] F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [5] A. Hermans, L. Beyer, and B. Bastian, "In defense of the triplet loss for person re-identification," *arXiv preprint arXiv:1703.07737*, 2017.
- [6] L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi, and P. H. Torr, "Fully-convolutional siamese networks for object tracking," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 850–865.
- [7] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a 'Siamese' time delay neural network," in *Adv. Neural Inf. Process. Syst. (NIPS)*, 1993, pp. 737–744.
- [8] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4685–4694.

EK -1

Görüntü benzerliği araması için metrik öğrenme CIFAR-10

Kodlama işlemi Google Colab üzerinden gerçekleştirilmiştir. Jupyter Notebook üzerinde kütüphanelerin kullanımında sorunların çıkma olasılığı, veri tabanlarına kolay bir şekilde ulaşım ve drive dosyalarının kolaylıkla kullanılabilmesi amacıyla bu kodlama ortamı kullanılmıştır. Aşağıdaki linkten Google Colab ortamına erişebilirsiniz. Çalışmanızda kolaylıklar dilerim. Defne BENLİY

Paylaşılmış Google Colab drive linki

<https://colab.research.google.com/drive/14gYdWFAIKKx5hGBiNFXL3bfNMmC0Ux3U?usp=sharing>

import os

os.environ["KERAS_BACKEND"] = "tensorflow"

import random

import matplotlib.pyplot as plt

import numpy as np

import tensorflow as tf

from collections import defaultdict

from PIL import Image

from sklearn.metrics import ConfusionMatrixDisplay

import keras

from keras import layers

from keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train.astype("float32") / 255.0

y_train = np.squeeze(y_train)

x_test = x_test.astype("float32") / 255.0

y_test = np.squeeze(y_test)

height_width = 32

def show_collage(examples):

 box_size = height_width + 2

 num_rows, num_cols = examples.shape[:2]

 collage = Image.new(

 mode="RGB",

 size=(num_cols * box_size, num_rows * box_size),

 color=(250, 250, 250),

)

for row_idx **in** range(num_rows):

for col_idx **in** range(num_cols):

```
array = (np.array(examples[row_idx, col_idx]) * 255).astype(np.uint8)
collage.paste(
    Image.fromarray(array), (col_idx * box_size, row_idx * box_size)
)
```

Double size for visualisation.

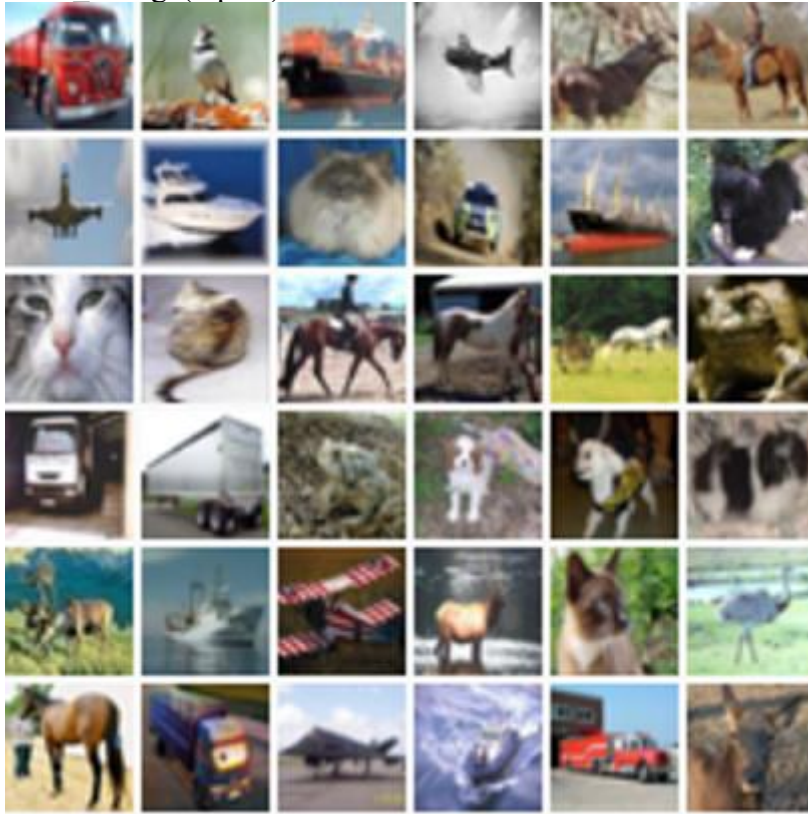
```
collage = collage.resize((2 * num_cols * box_size, 2 * num_rows * box_size))
return collage
```

```
# Show a collage of 5x5 random images.
```

```
sample_idx = np.random.randint(0, 50000, size=(6, 6))
```

```
inputs = x_train[sample_idx]
```

```
show collage(inputs)
```



Yukarıdaki görseller için tanımlanmış olan cifar10 indeks değerleri (Örnek 3=Kedi)

```
outputs = y_train[sample_idx]
```

```
print (outputs)
```

[[9 2 8 0 4 7]]

 $[0\ 8\ 3\ 1\ 8\ 5]$

[3 3 7 7 7 6]

[9 9 6 5 5 5]

 $[4\ 8\ 0\ 4\ 3\ 2]$

[7 9 0 8 9 4]]

Metrik öğrenmede eğitim verisi doğrudan etiketli çiftler yerine, rastgele seçilen bir çapa (anchor) ve aynı sınıftan bir pozitif örnek arasındaki benzerlik ilişkisi üzerinden

kurgulanır. Bu süreci yürütmek için sınıfları ilgili örneklerle eşleştiren bir dizin yapısı oluşturulur ve eğitim sırasında veriler bu yapıdan örneklenerek kullanılır.

```
class_idx_to_train_idx = defaultdict(list)
for y_train_idx, y in enumerate(y_train):
    class_idx_to_train_idx[y].append(y_train_idx)

class_idx_to_test_idx = defaultdict(list)
for y_test_idx, y in enumerate(y_test):
    class_idx_to_test_idx[y].append(y_test_idx)
Kedi görsellerinin olduğu indeksler için 3 sayısı (ilk 5 tanesini gösterebilirsin)
print(class_idx_to_train_idx[3][:5])
```

Eğitim sürecinde her sınıftan birer adet olmak üzere oluşturulan (çapa, pozitif) çiftleri, aynı kümedeki diğer örneklerden uzaklaştırılıp birbirine yaklaştırılmaya çalışılır. Bu yöntemde veri paketinin (batch) boyutu doğrudan sınıf sayısına bağlıdır; örneğin CIFAR-10 veri kümesi için bu sayı 10 olarak belirlenir.

```
num_classes = 10
```

```
class AnchorPositivePairs(keras.utils.Sequence):
    def __init__(self, num_batches):
        super().__init__()
        self._num_batches = num_batches

    def __len__(self):
        return self._num_batches

    def __getitem__(self, _idx):
        x = np.empty((2, num_classes, height_width, height_width, 3), dtype=np.float32)
        for class_idx in range(num_classes):
            examples_for_class = class_idx_to_train_idx[class_idx]
            anchor_idx = random.choice(examples_for_class)
            positive_idx = random.choice(examples_for_class)
            while positive_idx == anchor_idx:
                positive_idx = random.choice(examples_for_class)
            x[0, class_idx] = x_train[anchor_idx]
            x[1, class_idx] = x_train[positive_idx]
        return x
```

Oluşturulan veri paketini bir kolaj yardımıyla görselleştirmek mümkündür. Bu görselde üst satır, on farklı sınıftan rastgele seçilmiş çapa (anchor) örneklerini, alt satır ise bunlara karşılık gelen on pozitif örneği gösterir. Uçak, Araba, Kuş, Kedi, Geyik, Köpek, Kurbaa, At, Gemi, Kamyon

```
inputs = next(iter(AnchorPositivePairs(num_batches=1)))
```

```
show_collage(inputs)
```




Tanımlanan bu modelin eğitim adımında (train_step), öncelikle çapa ve pozitif örneklerin sayısal vektör karşılıkları (embedding) oluşturulur. Ardından, bu vektörlerin birbirleriyle olan nokta çarpımları (dot product) hesaplanarak, benzerlik skorlarını belirleyen bir softmax katmanı için girdi (logits) olarak kullanılır.

```
class EmbeddingModel(keras.Model):
```

```
    def train_step(self, data):
```

```
        # Not: Açık soruna yönelik geçici çözüm kaldırılacaktır.
```

```
        if isinstance(data, tuple):
```

```
            data = data[0]
```

```
        anchors, positives = data[0], data[1]
```

```
        with tf.GradientTape() as tape:
```

```
            # Hem çapraz hem de pozitif değerleri model üzerinden geçirir.
```

```
            anchor_embeddings = self(anchors, training=True)
```

```
            positive_embeddings = self(positives, training=True)
```

```
            # Çapa noktaları ve pozitif noktalar arasındaki kosinüs benzerliğini hesaplar.
```

```
            # Normalleştirildikleri için bu sadece ikili nokta çarpımlarıdır.
```

```
            benzerlik = keras.ops.einsum(
                "ae,pe->ap", anchor_embeddings, positive_embeddings
            )
```

```
            # Bunları logit olarak kullanmayı planladığımız için, bunları bir katsayıyla ölçeklendiriyoruz.
```

```
            # Bu değer normalde bir hiper parametre olarak seçilir. temperature olarak geçiyor literatürde.
```

```
            katsayi = 0.2
```

```
            benzerlik /= katsayi
```

```
            # Çapa/pozitif çiftlere karşılık gelen ana diyagonal değerlerin yüksek olması isteniyor.
```

```
            # Bu kayıp, çapa/pozitif çiftler için gömülü vektörleri birbirine yaklaştıracak ve
```

```
            # diğer tüm çiftleri birbirinden uzaklaştıracaktır.
```

```
            sparse_labels = keras.ops.arange(num_classes)
```

```
            loss = self.compute_loss(y=sparse_labels, y_pred= benzerlik)
```

```
            # Eğitimleri hesaplar ve optimizasyon algoritması aracılığıyla uygular.
```

```

gradients = tape.gradient(loss, self.trainable_variables)
self.optimizer.apply_gradients(zip(gradients, self.trainable_variables))

# Güncellemeleri yapar ve ölçüm değerlerini (özellikle kayıp değeriyle ilgili olanı)
döndürür.
for metric in self.metrics:

    if metric.name == "loss":
        metric.update_state(loss)
    else:
        metric.update_state(sparse_labels, benzerlik)

return {m.name: m.result() for m in self.metrics}

```

Görüntüleri vektör uzayına aktaran bu model; evrişimli katmanlar (CNN), global havuzlama ve doğrusal bir projeksiyon katmanından oluşur. Benzerlik hesaplamasını kolaylaştırmak için vektörler normalize edilir ve model yapısı basitlik adına küçük tutulmuştur.

```

inputs = layers.Input(shape=(height_width, height_width, 3))
x = layers.Conv2D(filters=32, kernel_size=3, strides=2, activation="relu")(inputs)
x = layers.Conv2D(filters=64, kernel_size=3, strides=2, activation="relu")(x)
x = layers.Conv2D(filters=128, kernel_size=3, strides=2, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)

embeddings = layers.Dense(units=8, activation=None)(x)
embeddings = layers.UnitNormalization()(embeddings)

model = EmbeddingModel(inputs, embeddings)

```

Son olarak eğitimi başlatıyoruz. Google Colab (CPU 5 dakika) GPU örneğinde 1 dakika sürüyor.

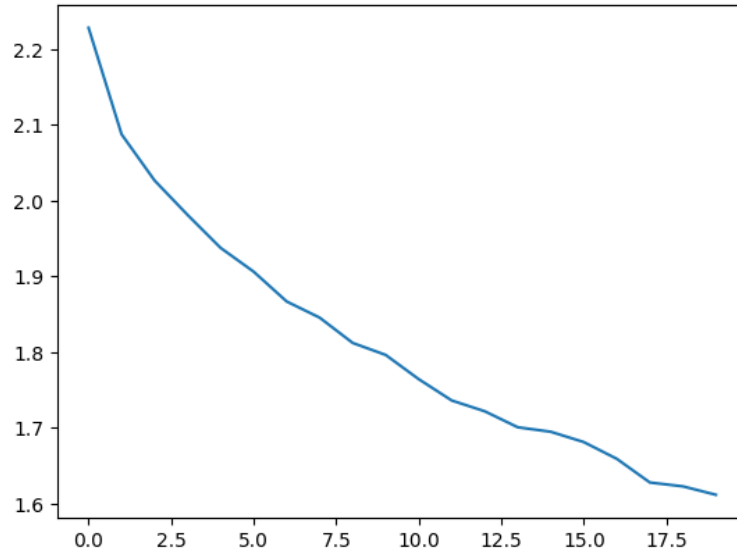
```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
)

history = model.fit(AnchorPositivePairs(num_batches=1000), epochs=20)

plt.plot(history.history["loss"])
plt.show()

```



Öncelikle test kümesini gömüyoruz ve tüm yakın komşuları hesaplıyoruz. Gömme işlemlerinin birim uzunlukta olduğunu hatırlarsak, nokta çarpımları yoluyla kosinüs benzerliğini hesaplayabiliriz.

```
near_neighbours_per_example = 10
```

```
embeddings = model.predict(x_test)
gram_matrix = np.einsum("ae,be->ab", embeddings, embeddings)
near_neighbours = np.argsort(gram_matrix.T)[:,-(near_neighbours_per_example + 1) :]
```

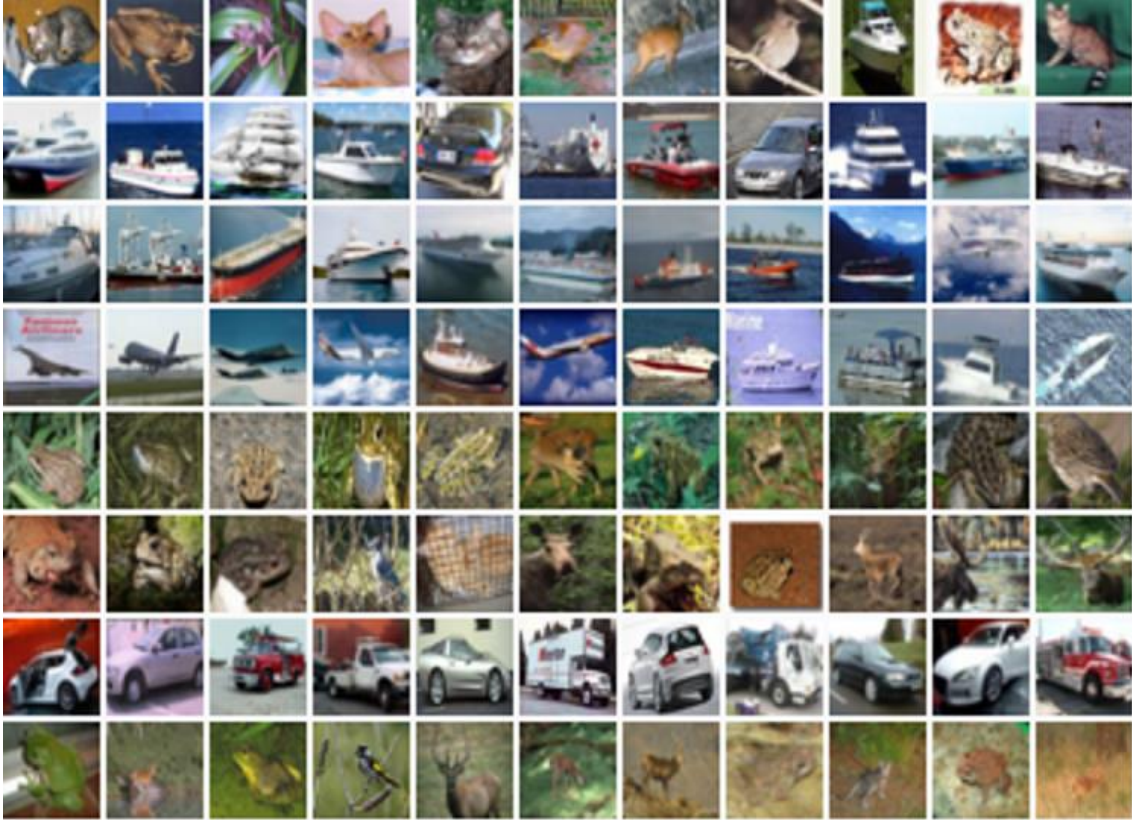
313/313 ————— **1s 2ms/step**

Gömülü vektörlerin (embeddings) başarısını görsel olarak denetlemek için, rastgele seçilen 8 örnek ve bunların en yakın komşularından oluşan bir kolaj oluşturuldu. Bu görselde ilk sütun hedef görüntüyü, sonraki 10 sütun ise benzerlik sırasına göre dizilmiş en yakın komşuları temsil eder.

```
num_collage_examples = 8
```

```
examples = np.empty(
    (
        num_collage_examples,
        near_neighbours_per_example + 1,
        height_width,
        height_width,
        3,
    ),
    dtype=np.float32,
)
for row_idx in range(num_collage_examples):
    examples[row_idx, 0] = x_test[row_idx]
    anchor_near_neighbours = reversed(near_neighbours[row_idx][:1])
    for col_idx, nn_idx in enumerate(anchor_near_neighbours):
        examples[row_idx, col_idx + 1] = x_test[nn_idx]
```

show_collage(examples)



Performans analizi için her sınıftan on örnek seçilip yakın komşularının aynı sınıfa ait olup olmadığına bakılarak bir karışıklık matrisi (confusion matrix) oluşturulur. Gözlemler sonucunda hayvan ve araç sınıflarının kendi içlerinde yüksek doğruluk sergilediği, hataların ise genellikle benzer kategoriler (hayvanların hayvanlarla, araçların araçlarla) arasında yaşandığı görülür.

```
confusion_matrix = np.zeros((num_classes, num_classes))
```

For each class.

```
for class_idx in range(num_classes):
```

Consider 10 examples.

```
example_idx = class_idx_to_test_idx[class_idx][:10]
```

```
for y_test_idx in example_idx:
```

And count the classes of its near neighbours.

```
for nn_idx in near_neighbours[y_test_idx][:1]:
```

```
nn_class_idx = y_test[nn_idx]
```

```
confusion_matrix[class_idx, nn_class_idx] += 1
```

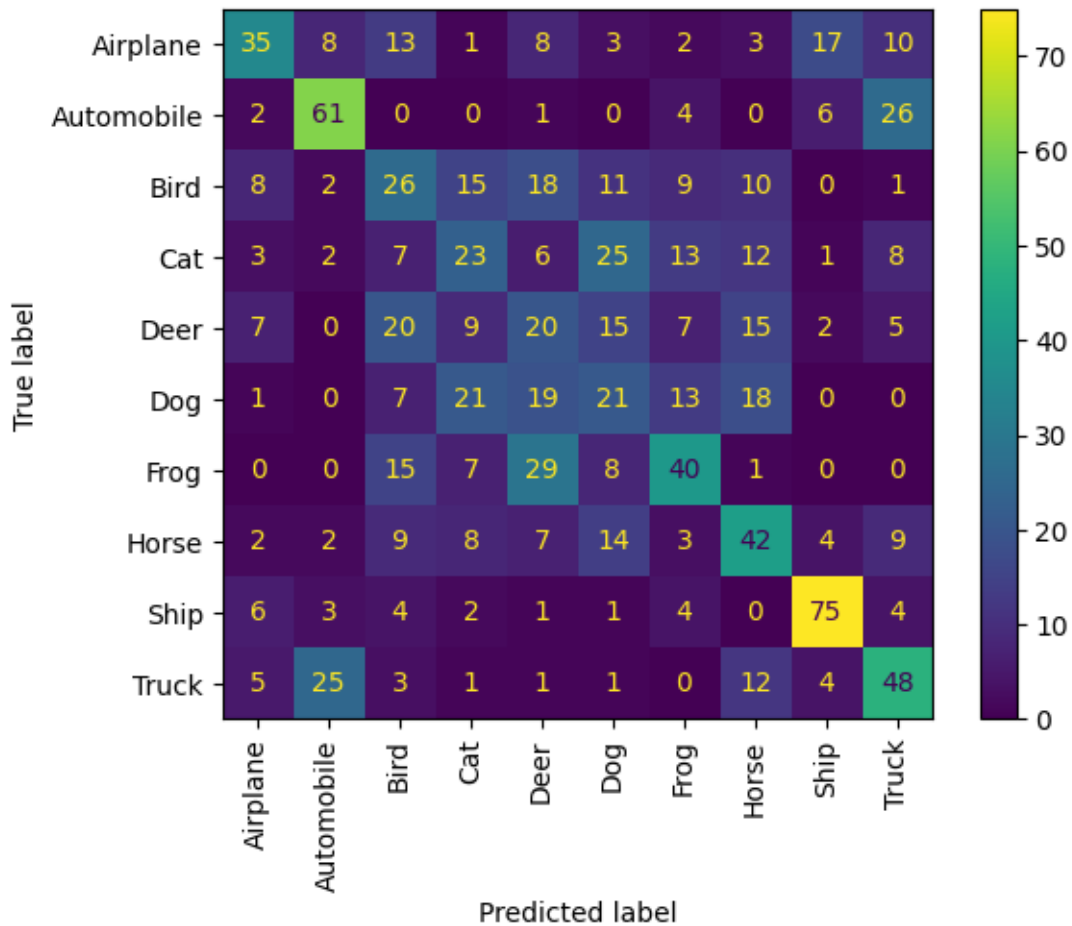
Display a confusion matrix.

```
labels = [  
    "Airplane",  
    "Automobile",  
    "Bird",
```

```

    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck",
]
disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix,
                               display_labels=labels)
disp.plot(include_values=True, cmap="viridis", ax=None, xticks_rotation="vertical")
plt.show()

```



```

from google.colab import files

```

```

print("Lütfen denemek istediğiniz resimleri seçin:")
uploaded = files.upload()

```

```

labels = [
    "Airplane (Uçak)", "Automobile (Otomobil)", "Bird (Kuş)", "Cat (Kedi)",
    "Deer (Geyik)", "Dog (Köpek)", "Frog (Kurbağa)", "Horse (At)",

```

```

    "Ship (Gemi)", "Truck (Kamyon)"
]

def process_and_predict(img_path, model, test_embeddings, y_test):
    img = Image.open(img_path).convert('RGB').resize((32, 32))
    img_array = np.array(img).astype('float32') / 255.0
    img_tensor = np.expand_dims(img_array, axis=0)

    target_embedding = model.predict(img_tensor, verbose=0)

    similarities = np.dot(target_embedding, test_embeddings.T)

    closest_idx = np.argmax(similarities)
    predicted_class = y_test[closest_idx]

    return img, labels[predicted_class]

test_embeddings = model.predict(x_test, verbose=0)
plt.figure(figsize=(10, 10))
num_uploaded_items = len(uploaded.items())

num_cols = int(np.ceil(num_uploaded_items / 2.0)) if num_uploaded_items > 0 else 1
num_rows = 2 if num_uploaded_items > 2 else 1 # Eğer 2'den fazla resim varsa 2 satır,
yoksa 1

for i, (filename, content) in enumerate(uploaded.items()):
    pil_img, result_text = process_and_predict(filename, model, test_embeddings, y_test)

    plt.subplot(num_rows, num_cols, i + 1)
    plt.imshow(pil_img)
    plt.title(f'Tahmin: {result_text}\n({filename})')
    plt.axis('off')

plt.tight_layout()
plt.show()

```

EK -2

Triplet Loss

Kodlama işlemi Google Colab üzerinden gerçekleştirilmiştir. Jupyter Notebook üzerinde kütüphanelerin kullanımında sorunların çıkma olasılığı, veri tabanlarına kolay bir şekilde ulaşım ve drive dosyalarının kolaylıkla kullanılabilmesi amacıyla bu kodlama ortamı kullanılmıştır. Aşağıdaki linkten Google Colab ortamına erişebilirsiniz. Çalışmanızda kolaylıklar dilerim. Defne BENLİY

https://colab.research.google.com/drive/1rvNqiYHYnFzI86ifLn70FNVqr7G_wQU-?usp=sharing

```
import numpy as np
import pandas as pd
import tensorflow as tf
import cv2
import matplotlib.pyplot as plt
import os
from sklearn.preprocessing import LabelEncoder
import zipfile
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from google.colab import files
from google.colab import drive
from tensorflow.keras import backend as K
```

```
drive.mount('/content/drive')
zip_path = '/content/drive/MyDrive/archive.zip'
unzip_dir = '/content/archive/'
os.makedirs(unzip_dir, exist_ok=True)
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(unzip_dir)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

```
for root, dirs, files in os.walk('/content/archive/'):
    for name in files:
        print(os.path.join(root, name))
```

```

for name in dirs:
    print(os.path.join(root, name))

.....>
!wget
https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_frontalface_default.xml -O haar.xml

haar.xml      100%[=====>] 908.33K  --.-KB/s   in 0.03s

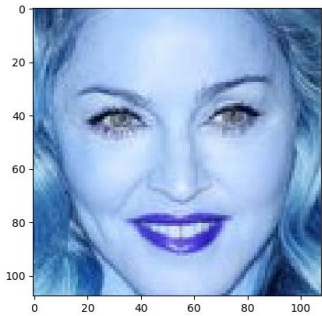
2026-04-25 17:31:35 (31.3 MB/s) - 'haar.xml' saved [930127/930127]

haar_cascade_path = '/content/haar.xml'
face_cascade = cv2.CascadeClassifier(haar_cascade_path)
image_path =
"/content/archive/data/train/madonna/httpiamediaimdbcomimagesMMVBMTANDQ
NTAxNDVeQTJeQWpwZBbWUMDIMjQOTYVUXCRALjpg.jpg"
f = cv2.imread(image_path)

faces = face_cascade.detectMultiScale(f,1.3,5)
print(faces)
for x,y,w,h in faces:
    plt.imshow(f[y:y+h, x:x+w])

[[ 50  50 108 108]]

```



```

dirs = "/content/archive/data/train/"
img_size = 60
data = []
for name in os.listdir(dirs):
    for f in os.listdir(dirs+name):
        f = cv2.imread(os.path.join(dirs+name, f))
        faces = face_cascade.detectMultiScale(f,1.3,5)
        for x,y,w,h in faces:

```



```

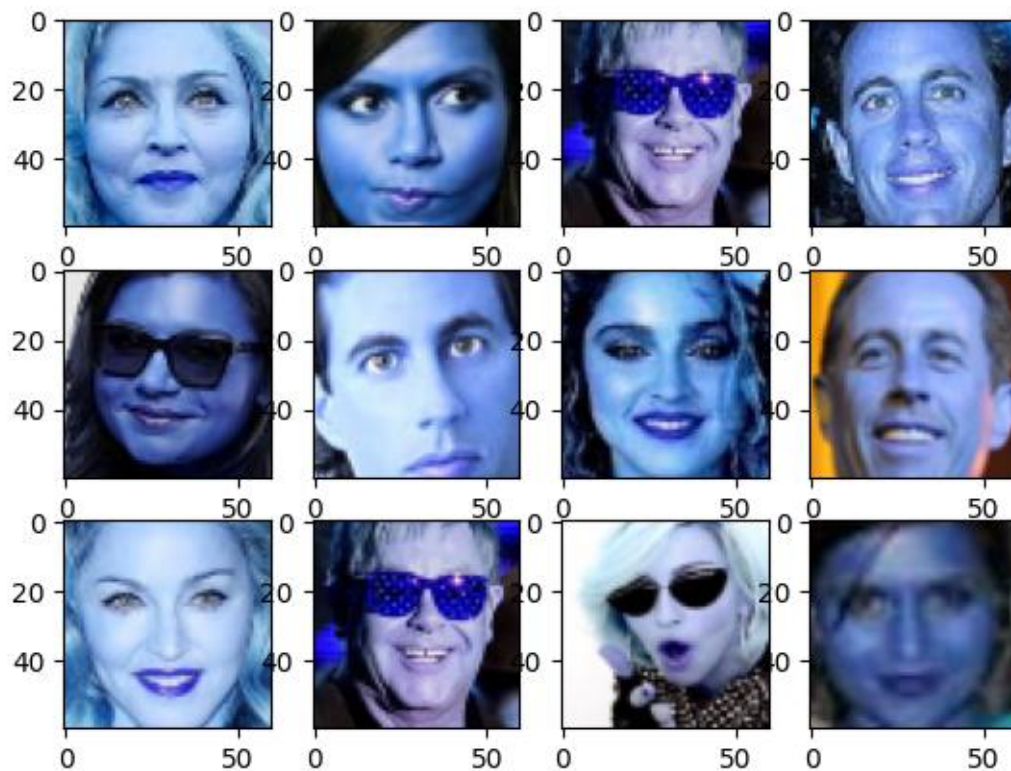
img = f[y:y+h, x:x+w]
img = cv2.resize(img, (img_size,img_size))
data.append((img, name))

df = pd.DataFrame(data, columns=["image", "name"])
print("Length:",len(df))

Length: 75

for i in range(12):
    row = df.iloc[np.random.randint(0, len(df)), :]
    plt.subplot(3,4,i+1)
    plt.imshow(row['image'])

```



```

dirs = "/content/archive/data/val/"
data = []
for name in os.listdir(dirs):
    for f in os.listdir(dirs+name):
        f = cv2.imread(os.path.join(dirs+name, f))
        faces = face_cascade.detectMultiScale(f,1.3,5)
        for x,y,w,h in faces:

```

```
img = f[y:y+h, x:x+w]
img = cv2.resize(img, (img_size,img_size))
data.append((img, name))
```

```
df_test = pd.DataFrame(data, columns=["image", "name"])
print("Test size: ", len(df_test))
```

Test size: 24

```
df_test.head()
```

	image	name
0	[[[56, 48, 49], [53, 48, 50], [52, 51, 53], [5...	jerry_seinfeld
1	[[[33, 30, 23], [30, 27, 22], [26, 24, 20], [2...	jerry_seinfeld
2	[[[102, 131, 171], [83, 112, 148], [51, 54, 54...	jerry_seinfeld
3	[[[5, 14, 4], [5, 14, 4], [5, 17, 5], [5, 21, ...	jerry_seinfeld
4	[[[24, 36, 78], [23, 35, 78], [23, 36, 76], [1...	jerry_seinfeld

```
le = LabelEncoder()
le.fit(df["name"].values)
```

```
LabelEncoder()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
x_train = list(df['image'].values)
x_train = np.array(x_train)
x_train = x_train/255
print(x_train.shape)
```

```
y_train = le.transform(df["name"].values)
print(y_train.shape)
```

```
(75, 60, 60, 3)
(75,)
```

```

x_test = list(df_test['image'].values)
x_test = np.array(x_test)
x_test = x_test/255
print(x_test.shape)

```

```

y_test = le.transform(df_test["name"].values)
print(y_test.shape)

```

```

(24, 60, 60, 3)
(24,)

```

```

people_num = len(np.unique(y_train))
people_num

```

```

5

```

```

def triplet_loss(y_true, y_pred, alpha = 0.2):
    total_lenght = y_pred.shape.as_list()[-1]
    anchor, positive, negative = y_pred[:,int(1/3*total_lenght)],
    y_pred[:,int(1/3*total_lenght):int(2/3*total_lenght)], y_pred[:,int(2/3*total_lenght):]

    pos_dist = tf.reduce_sum(tf.square(anchor - positive), axis=-1)
    neg_dist = tf.reduce_sum(tf.square(anchor - negative), axis=-1)
    basic_loss = pos_dist - neg_dist + alpha
    loss = tf.reduce_sum(tf.maximum(basic_loss,0.0))
    return loss

```

```

def generate_triplets(x, y, num_same, num_diff):
    anchor_images = np.array([]).reshape((-1,)+ x.shape[1:])
    same_images = np.array([]).reshape((-1,)+ x.shape[1:])
    diff_images = np.array([]).reshape((-1,)+ x.shape[1:])

    for i in range(len(y)):
        point = y[i]
        anchor = x[i]

        same_pairs = np.where(y == point)[0]
        same_pairs = np.delete(same_pairs , np.where(same_pairs == i))
        diff_pairs = np.where(y != point)[0]

```

```

same = x[np.random.choice(same_pairs,num_same)]
diff = x[np.random.choice(diff_pairs,num_diff)]

anchor_images = np.concatenate((anchor_images, np.tile(anchor, (num_same *
num_diff, 1, 1, 1) )), axis = 0)

for s in same:
    same_images = np.concatenate((same_images, np.tile(s, (num_same, 1, 1, 1) )),
axis = 0)
    diff_images = np.concatenate((diff_images, np.tile(diff, (num_diff, 1, 1, 1) )), axis
= 0)
return anchor_images, same_images, diff_images

anchor_images, same_images, diff_images = generate_triplets(x_train,y_train,
num_same= 10, num_diff=10)
print(anchor_images.shape, same_images.shape, diff_images.shape)

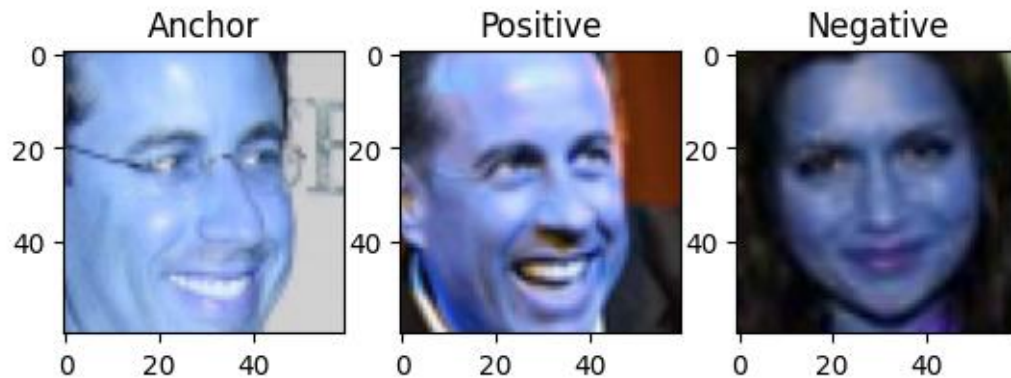
(7500, 60, 60, 3) (7500, 60, 60, 3) (7500, 60, 60, 3)

idx = 100
plt.subplot(1,3,1)
plt.imshow(anchor_images[idx])
plt.title('Anchor')

plt.subplot(1,3,2)
plt.imshow(same_images[idx])
plt.title('Positive')

plt.subplot(1,3,3)
plt.imshow(diff_images[idx])
plt.title('Negative')
print()

```



Building and Fitting the CNN Model

```
def get_model():
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding='same',
    input_shape=(img_size,img_size,3), activation='relu'))
    model.add(tf.keras.layers.Conv2D(128, kernel_size=3, strides=2, padding='same',
    activation='relu'))
    model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding='same',
    activation='relu'))
    model.add(tf.keras.layers.Conv2D(64, kernel_size=1, strides=2, padding='same',
    activation='relu'))
    model.add(tf.keras.layers.Flatten())

    model.add(tf.keras.layers.Dense(256, activation='relu'))
    model.add(tf.keras.layers.Dropout(0.1))
    model.add(tf.keras.layers.Dense(128))
    model.add(tf.keras.layers.Lambda(lambda x: K.l2_normalize(x, axis=1)))

    # model.summary() # Modeli ilk kez çağırdıktan sonra summary'yi çağırabilirsiniz
    return model
```

```
anchor_input = tf.keras.layers.Input((img_size, img_size, 3), name='anchor_input')
positive_input = tf.keras.layers.Input((img_size, img_size, 3), name='positive_input')
negative_input = tf.keras.layers.Input((img_size, img_size, 3), name='negative_input')
```

```
shared_dnn = get_model()
encoded_anchor = shared_dnn(anchor_input)
encoded_positive = shared_dnn(positive_input)
encoded_negative = shared_dnn(negative_input)
```

```
merged_vector = tf.keras.layers.concatenate([encoded_anchor, encoded_positive,
                                              encoded_negative], axis=-1, name='merged_layer')
```

```
model = tf.keras.Model(inputs=[anchor_input, positive_input, negative_input],
                        outputs=merged_vector)
model.summary()
model.compile(loss=triplet_loss, optimizer="adam")
```

```
Y_dummy = np.empty((anchor_images.shape[0],1))
model.fit([anchor_images, same_images, diff_images], y=Y_dummy, batch_size=128,
          epochs=10)
```

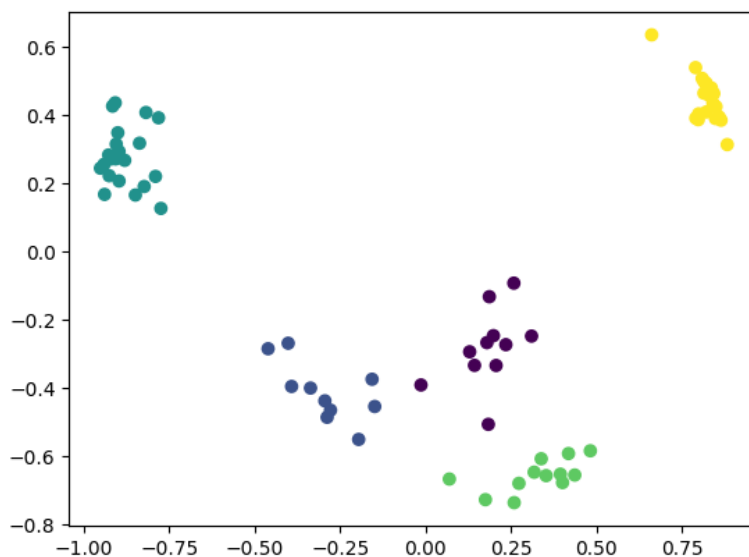
```
anchor_model = tf.keras.Model(inputs = anchor_input, outputs=encoded_anchor)
```

```
pred = anchor_model.predict(x_train)
pred.shape
```

```
3/3 ————— 1s 197ms/step
(75, 128)
```

```
pca = PCA(n_components=2)
pred_pca = pca.fit_transform(pred)
plt.scatter(pred_pca[:,0], pred_pca[:,1], c=y_train)
```

```
<matplotlib.collections.PathCollection at 0x7a52cc348200>
```



Face Recognition using KNN

```
def encode_image(model ,img):
    encode = model.predict(img.reshape((1,)+ img.shape))
    return encode
```

```
pred_x_train = anchor_model.predict(x_train)
```

3/3 ————— 0s 12ms/step

```
neigh = KNeighborsClassifier(n_neighbors=7)
neigh.fit(pred_x_train, y_train)
```

```
KNeighborsClassifier(n_neighbors=7)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

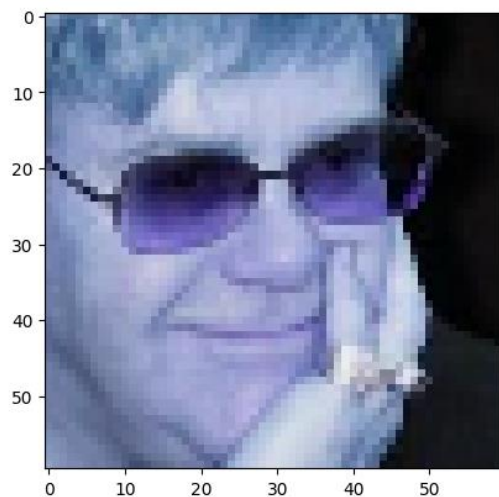
```
idx = 8
img = x_test[idx]
plt.imshow(img)
```

```
enc = encode_image(anchor_model, img)
pred = neigh.predict(enc)
print("Predicted name:",le.inverse_transform(pred))
print("Actual pred: ", le.inverse_transform(y_test[idx:idx+1]))
```

1/1 ————— 0s 374ms/step

```
Predicted name: ['madonna']
```

```
Actual pred: ['elton_john']
```



```

idx = 10
img = x_test[idx]
plt.imshow(img)

enc = encode_image(anchor_model, img)
pred = neigh.predict(enc)
print("Predicted name:", le.inverse_transform(pred))
print("Actual pred: ", le.inverse_transform(y_test[idx:idx+1]))

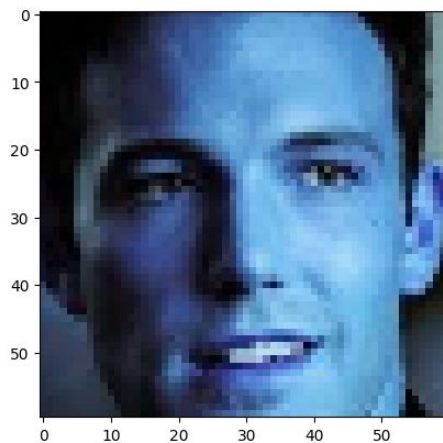
```

1/1 ————— 0s 37ms/step

```

Predicted name: ['ben_afflek']
Actual pred: ['ben_afflek']

```



```

from google.colab import files
uploaded = files.upload()

```

```

for fn in uploaded.keys():
    uploaded_image_path = fn
    print(f'User uploaded file "{uploaded_image_path}"')
    break

```

```
img = cv2.imread(uploaded_image_path)
```

```

if img_uploaded is None:
    print(f'Error: Could not load image from {uploaded_image_path}')
else:
    faces_uploaded = face_cascade.detectMultiScale(img, 1.3, 5)

    if len(faces_uploaded) == 0:
        print("No faces detected in the uploaded image.")

```



```

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert to RGB for
display
plt.title('No Face Detected')
plt.axis('off')
plt.show()
else:
    print(f'Found {len(faces_uploaded)} face(s) in the uploaded image.')
    x, y, w, h = faces_uploaded[0]

    face_crop = img[y:y+h, x:x+w]
    face_resized = cv2.resize(face_crop, (img_size, img_size))
    face_normalized = face_resized / 255.0
    plt.imshow(cv2.cvtColor(face_normalized.astype(np.float32),
cv2.COLOR_BGR2RGB)) # Convert to RGB for display
    plt.title('Detected Face')
    plt.axis('off')
    plt.show()

    print(f'Normalized face shape: {face_normalized.shape}')
    enc = encode_image(anchor_model, face_normalized)
    pred = neigh.predict(enc)
    print("Predicted name:", le.inverse_transform(pred))

```

Saving sei.jpg to sei (3).jpg
User uploaded file "sei (3).jpg"
Found 1 face(s) in the uploaded image.

Detected Face



Normalized face shape: (60, 60, 3)

1/1 ————— 0s 37ms/step

Predicted name: ['jerry_seinfeld']

```
pred_x_test = anchor_model.predict(x_test)
pred = neigh.predict(pred_x_test)
print("Accuracy ->", np.sum(pred == y_test) / len(pred))
```

```
1/1 _____ 1s 837ms/step
Accuracy -> 0.8666666666666666
```